Hardware Root of Trust in RISC-V Systems: Analysis and Evaluation of Open Source Approaches

Pau Romaguera Palomé

December 9, 2025

Resum– L'evolució ràpida del hardware d'altes prestacions ha transformat la indústria dels semiconductors, possibilitant avenços en intel·ligència artificial (IA) i computació d'altes prestacions (HPC). A mesura que aquests sistemes esdevenen més sofisticats, garantir la seguretat del xip és essencial per protegir les dades que processen. Aquesta tesi avalua funcionalitats de seguretat de codi obert en sistemes basats en RISC-V mitjançant proves adversàries, centrant-nos en mecanismes de protecció de memòria com la protecció física de memòria (PMP) i la PMP millorada (ePMP), així com en la verificació criptogràfica dins del Secure Boot. Prenent com a cas d'estudi el System on Chip (SoC) anomenat Earlgrey d'OpenTitan [1] [2] [3], afeblim o eliminem proteccions seleccionades per quantificar la contribució de cada mecanisme i derivar recomanacions pràctiques per enfortir dissenys de Hardware Root of Trust (HRoT).

Paraules clau– Computació d'altes prestacions (HPC), plataformes RISC-V, intel·ligència artificial (IA), sistemes arrelats en hardware (HRoT), Protecció Física de Memòria (PMP)

Abstract– The rapid evolution of high-performance hardware has transformed the semiconductor industry, enabling advances in artificial intelligence (AI) and high-performance computing (HPC). As these systems grow more sophisticated, ensuring chip security is essential to protect the data they process. This thesis evaluates open-source security features in RISC-V-based systems using adversarial testing, focusing on memory protection mechanisms such as Physical Memory Protection (PMP) and enhanced PMP (ePMP) and cryptographic verification in secure boot. Using OpenTitan's [1] [2] [3] Earlgrey System on Chip (SoC) as the case study, we weaken or remove selected protections to quantify each mechanism's contribution and derive practical recommendations for strengthening hardware root of trust (HRoT) designs.

Keywords– High Performance Computing (HPC), RISC-V platforms, artificial intelligence (AI), System on Chip (SoC), Hardware Root of Trust (HRoT), Physical Memory Protection (PMP)

1 Introduction - Context of the thesis

Modern semiconductor designs increasingly embed security features directly in hardware so that sensitive operations can be protected with minimal performance penalties.

The open RISC-V instruction set architecture (ISA) [6] accelerates this shift by enabling community-driven security extensions and cryptographic instructions that, together with dedicated accelerators and on-chip hardware roots of trust (HRoT), provide robust security with low overhead, safeguarding data and code integrity for end users.

In this thesis, we focus on the HRoT model in RISC-V systems, using OpenTitan as the primary case study [1]. Our analysis centers on the mechanisms that underpin a secure boot chain, particularly Physical Memory Protection (PMP) [8][9] and its enhanced variant ePMP [5], both of which enforce memory access controls from the earliest boot stages. By evaluating how these mechanisms contribute to overall system security, we aim to derive insights and recommendations for designing effective HRoT imple-

• E-mail de contacte: pauromaguera1@gmail.com
• Mención: Enginyeria de Computadors
• Treball tutoritzat per: Marta Baldrís Calmet (Openchip and Software Technologies S.L.) Raimon Casanova Mohr(Departament de Microelectrònica i Sistemes Electrònics)
• Curs 2025/26

mentations in RISC-V SoCs.

HRoT functionality is typically integrated within a System-on-Chip (SoC) and may include a dedicated core alongside hardened peripherals and memories to minimize the attack surface and resist software-based compromise. In this context, a clear separation of roles across boot stages, each with narrowly scoped privileges, enables more robust and reliable protection. In the remainder of this thesis we experimentally stress-test these mechanisms on OpenTitan's Earlgrey SoC to assess their effectiveness against realistic adversaries and identify potential weaknesses or misconfigurations.

At the same time, advanced RISC-V designs use a general-purpose management core to orchestrate secure boot, power management, and low-level control for larger compute subsystems such as AI accelerators. These management cores often implement only the base PMP extension rather than ePMP. Understanding how security properties change when enhanced features such as machine-mode whitelisting (MMWP) are absent is therefore relevant beyond OpenTitan itself, in particular for heterogeneous SoCs where a relatively simple control core is expected to establish a strong hardware root of trust for more complex accelerators.

[Safe to mention high level company objectives like this?]

2 Objectives

The main objective of this thesis is to analyze and empirically prove the level of security of the open-source Earlgrey SoC from OpenTitan. In particular, the contribution of certain security mechanisms will be examined, such as Secure Boot and enhanced Physical Memory Protection (ePMP) [5], to verify how each function contributes to the system's overall security.

To stress-test these mechanisms, we will use ablation tests. This involves intentionally modifying or removing specific security policies or functions in order to assess how vulnerable the SoC becomes to certain attack vectors. In addition, we compare the effective behaviour of OpenTitan's ePMP-based configuration against a baseline that relies only on standard PMP semantics. This allows us to characterise which guarantees depend critically on ePMP-specific features and which can be approximated on platforms where a general-purpose control core, responsible for secure boot and power management of larger RISC-V subsystems, implements only base PMP.

In doing so, we can identify which measures are most critical for protecting the system and establish a hierarchy of defenses based on their actual effectiveness.

3 Project Management Methodology and Planning

The project has been organized according to the Scrum methodology. The duration of tasks are planned in two-week iterative cycles, called sprints. Each task is expected to be completed within a sprint or, when necessary, decomposed into smaller sub-tasks to fit the two-week cycle. In addition, daily standup meetings are held with Openchip's "Security Features" team to share progress and issues, while Sprint Reviews are conducted to evaluate results, and retrospectives identify opportunities for improvement.

As mentioned earlier, each task has been structured to fit biweekly cadence, although some larger tasks have spanned two sprints, depending on issues encountered during execution. The timeline can be seen in the initial Gantt chart in Fig. 1, which shows the sequence and duration of the main activities.

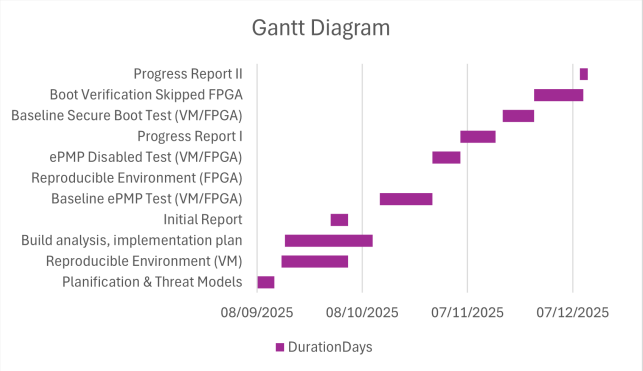

Fig. 1: Gantt diagram corresponding to the task execution of the project.

4 State of the Art

[NEED REVISE:] Within the scope of this project, the development relies on OpenTitan [1], the first open-source hardware project to reach the market. It is a community-driven initiative supported by several organizations, including lowRISC [11], ETH Zurich [12], Nuvoton [13], Google [14], Seagate [14], Western Digital [16], among others. The company lowRISC is actively developing the processor that powers OpenTitan, known as Ibex, while Nuvoton has taken the SoC to its first tape-out (physical production), with the aim of integrating it as a HRoT in upcoming Google Chromebooks[17].

Reaching truly open-source hardware is exceptionally significant. Unlike proprietary hardware, open hardware allows every component of the design to be publicly audited, verified, and improved. This transparency is particularly crucial in a security context, as it reduces the risk of hidden vulnerabilities, and makes it far more difficult to insert hardware Trojans[18], that is, malicious modifications hidden within circuits that could compromise the system, like back doors.

In a world where attacks are increasingly sophisticated and where the technological supply chain may be vulnerable, open-source hardware introduces a new paradigm: trust is not blindly delegated to a single manufacturer, but is instead built collectively through open review and the diversity of contributors involved. This strengthens the foundational security of the system and promotes a more resilient ecosystem against potential threats.

The development of this type of hardware would not have been possible without the prior advances of the RISC-V architecture, whose open and modular design has enabled the community to experiment with, extend, and validate new functionalities, thus laying the groundwork for ambitious

projects such as OpenTitan. [further revising:] The development of this type of hardware would not have been possible without the prior advances of the RISC-V architecture, whose open and modular design has enabled the community to experiment with, extend, and validate new functionalities, thus laying the groundwork for ambitious projects such as OpenTitan.

Beyond the processor core, OpenTitan's Earlgrey SoC integrates a number of specialised security blocks, including cryptographic engines (for example AES, HMAC, KMAC and OTBN, the OpenTitan Big Number accelerator for public-key cryptography), an entropy source and random number generator, a key manager, and a life-cycle controller that governs provisioning and debug access. Together with carefully partitioned non-volatile memory and peripheral isolation, these components implement a comprehensive hardware root of trust.

In this thesis we focus on the parts of Earlgrey that are directly involved in the early boot chain and memory isolation, namely the Ibex core with ePMP, the ROM and flash layout, and the associated boot firmware. Other security features such as the key manager and cryptographic accelerators are acknowledged but remain outside the scope of our experimental evaluation.

5 Threat Model

We assume an adversary with physical but non-invasive access to the device, such as a device owner, service provider, or unauthorised third party. The attacker can:

- interact with debug and test interfaces, peripheral ports, and external memory,
- read or modify firmware images stored in non-volatile memory,
- apply non-invasive fault mechanisms (for example, clock or voltage glitches) during early boot.

The attacker's primary goal is to bypass secure boot, for instance by executing unsigned or tampered firmware, or by redirecting control flow to unverified code. Invasive hardware attacks that require decapsulating the chip or probing internal wires are out of scope. This simplified model is aligned with the OpenTitan Lightweight Threat Model [19], but concentrates on non-invasive attacks against the secure-boot chain. Gaining the ability to run arbitrary firmware is particularly attractive for an attacker because it effectively disables all higher-level protections implemented above the root of trust and enables persistent, stealthy compromise of the platform. Real-world incidents such as the Stuxnet[**?**] malware, which targeted industrial control systems by manipulating low-level code in programmable logic controllers, illustrate how control over early-stage or firmware-level execution can be used to cause long-term, hard-to-detect damage in critical infrastructures.

6 System Under Test: Secure Boot

OpenTitan's secure boot chain ensures that every piece of firmware executed on the device is both authentic (signed by an approved key) and untampered (the signed hash matches with the firmware hash). The booting sequence is as follows:

OpenTitan's secure boot chain ensures that every piece of firmware executed on the device is both authentic (signed by an approved key) and untampered (the signed hash matches the firmware hash). Conceptually, it follows the familiar RISC-V boot pattern of a small immutable ROM that authenticates the next stage, which in turn authenticates later, more feature-rich firmware (for example, BL0, OpenSBI, and ultimately an operating system in a full SoC).

In OpenTitan, the immutable mask ROM and the ROM_EXT stage are signed by the *Silicon Creator* (the manufacturer), while subsequent stages such as BL0 are signed by the *Silicon Owner* (the device owner). For this thesis, we focus on the transition from ROM to ROM_EXT, because this is where the hardware root of trust and ePMP configuration first gate execution of mutable flash-resident code.

6.1 Boot Stages in OpenTitan

1. **ROM (Stage 0)** The first code executed on an OpenTitan SoC resides in an immutable mask ROM embedded directly in silicon. As this component cannot be updated post-manufacture, it constitutes the hardware root of trust and is responsible for authenticating all subsequent firmware stages. Upon power on, the processor begins execution at the ROM, which establishes a minimal trusted execution environment by configuring the enhanced Physical Memory Protection (ePMP) so that only the ROM region is executable. It then performs essential early initialization, including setting up the C runtime and enabling a small subset of peripherals.

   The ROM reads a manifest structure from flash, which specifies the location of the next boot stage (ROM_EXT), selects the appropriate RSA-3072 public key modulus for verification, and provides a cryptographic signature over the ROM_EXT image together with a set of usage constraints. The ROM computes a SHA-256 digest over the selected usage-constraint values concatenated with the ROM_EXT contents and verifies the signature using the public key indicated in the manifest. If verification succeeds, the ROM reconfigures ePMP to allow execution of the ROM_EXT flash region and transfers control. If verification fails, the boot process halts to prevent execution of untrusted code.

2. **ROM_EXT (Stage 1)**

   ROM_EXT ("ROM extension") is stored in flash and can be updated by the Silicon Creator, so its integrity is critical to the entire boot chain. ROM_EXT mirrors the core responsibility of the ROM: it authenticates the next stage in the boot sequence, known as BL0. To do so, it parses the BL0 manifest, computes a SHA-256 hash over the BL0 image, and verifies the accompanying RSA-3072 signature using the public key specified in the manifest. Only upon successful verification does ROM_EXT adjust the ePMP configuration to permit execution from the BL0 flash region and transfer control.

3. **BL0 (Stage 2)** BL0 is the first owner-controlled firmware stage. It performs platform-specific initialisation (for example, setting up buses and relocating larger firmware into SRAM) and then authenticates and boots the higher-level software stack (such as OpenSBI, bootloader, and operating system) following a similar ”verify before execute” principle. BL0 is not tested directly in our experiments, therefore it is included here only to show where the ROM→ROM_EXT execution handoff fits in the wider boot chain.

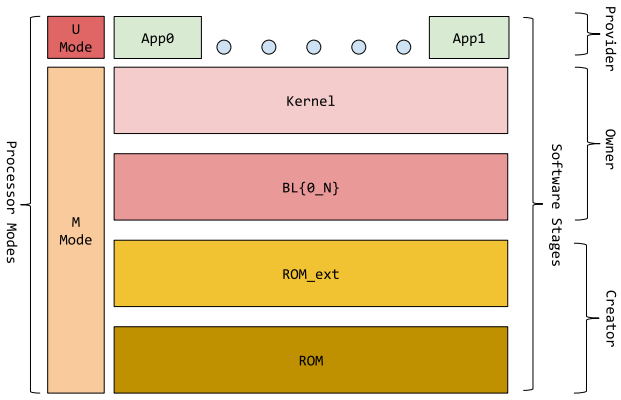

Fig. 2: Secure Boot Diagram

6.2 Memory Protection

OpenTitan’s 32-bit Ibex RISC-V core implements the enhanced Physical Memory Protection extension (ePMP). We first describe the base RISC-V Physical Memory Protection (PMP) in detail and then explain how ePMP strengthens early boot security and machine mode enforcement on this SoC.

6.2.1 PMP

To support secure processing and to contain software faults, it is often necessary to restrict which physical addresses software can access. The Physical Memory Protection (PMP) unit provides per-hart controls - where a hart refers to a “hardware thread” or an independent execution pipeline - that allow machine mode to specify access permissions (read, write, execute) for defined physical memory regions. By configuring these PMP entries, the system can enforce fine-grained isolation and determine which areas of the physical address space are accessible to lower-privileged software, thus improving overall memory safety and robustness.

Different RISC-V cores may implement varying PMP entry configurations. For instance, OpenTitan’s Ibex core uses 16 PMP entries in total, meaning sixteen memory regions to which we can enforce certain access modes, although different implementations differ in rule number. The standard uses two kinds of control status registers (CSR) to implement protection, namely:

6.2.1.1 pmpaddrN registers store the address fields that define the boundaries of PMP enforced memory regions.

TABLE 1: ONE-BYTE LAYOUT OF A `PMPCFG` ENTRY

Bits	Field	Allowed	Meaning / effect
7	L	0, 1	1: lock entry and enforce permissions on M-mode; 0: not locked (base PMP enforces on S/U only).
6–5	(reserved)	must be 0	Reserved; writes should be zero (WARL).
4–3	A[1:0]	00, 01, 10, 11	Address-matching mode.
2	X	0, 1	Execute permission for the matched region.
1	W	0, 1	Write permission for the matched region.*
0	R	0, 1	Read permission for the matched region.

* The combination R=0 and W=1 is reserved in PMP; implementations treat it as disallowing writes (i.e., equivalent to R=0, W=0).

The number of `pmpaddr` registers always matches the number of PMP entries implemented by the core. Importantly, the width of these registers is not determined by the CPU’s XLEN (32 or 64 bit). Rather, it is defined by the physical address width supported by the implementation. For example, an RV32 system that implements the Sv32 **??** virtual-memory scheme must support 34-bit physical addresses, so its `pmpaddr` registers expose the field `address[33:2]` as a WARL (Write Any, Read Legal) value. In systems without a memory management unit (MMU), such as OpenTitan’s Ibex core, therefore without virtual memory, the physical address space is only 32 bits wide, so the upper address bits simply read as zero and any writes to them are ignored according to WARL semantics.

6.2.1.2 pmpcfgN Each `pmpcfg` CSR is four bytes wide and encodes four PMP entries (one byte per entry). Concretely:

1. pmpcfg0: holds configurations of pmpaddr0...3
2. pmpcfg1: holds configurations of pmpaddr4...7
3. pmpcfg2: holds configurations of pmpaddr8...11
4. pmpcfg3: holds configurations of pmpaddr12...15

As stated above, each byte in a `pmpcfg` register encodes the configuration for one PMP entry. This byte specifies the read, write, and execute permissions (R/W/X), selects the address-matching mode via the `A` field (*OFF*, *TOR*, *NA4*, or *NAPOT*), and includes the lock bit `L`, which, when set, locks the entry until reset and causes M-mode to enforce the permissions. Bits [6:5] are reserved and must be written as zero. Together with the corresponding `pmpaddr[i]`, this configuration defines the protected region and its allowed accesses.

6.2.1.3 PMP address-matching modes (A field) These are the address-matching modes defined by RISC-V PMP. Each mode trades off granularity versus the number of PMP entries consumed to describe a region.

1. **00: OFF** The entry is disabled and defines no region. Its permissions are ignored. OFF entries can still serve as the lower bound for a following TOR region (via their `pmpaddr` value).

2. **01: TOR (Top of Range)** Defines a half-open interval [*lower*, *upper*). The *upper* bound is taken from the current entry's `pmpaddr[i]`, while the *lower* bound is the previous entry's `pmpaddr[i-1]` (or zero for `i=0`). TOR is convenient when the final size is only known after linking (e.g., the end of a program image). Note that TOR typically consumes *two* entries (one to hold the lower bound, one to hold the upper bound and permissions).

3. **10: NA4 (Naturally Aligned 4 bytes)** Protects a single 4-byte word at the address given by `pmpaddr[i]`. Useful to cover very small holes such as a stack guard word or a narrow MMIO register. It consumes a single entry but offers minimal coverage.

4. **11: NAPOT (Naturally Aligned Power of Two)** Covers a memory space that is aligned to a size of a power of two with just a single entry. This is ideal when the block size is known and aligned, for instance, a 16 KiB ROM or a 64 KiB flash partition.

6.2.1.4 The `mcause` CSR is a Machine-mode control and status register (CSR) that records the cause of the most recent trap taken into M-mode. A trap is either an exception (illegal instruction or a protection fault), or an interrupt (like timers or external event). In the context of this work, PMP is the primary source of protection-fault exceptions: attempting to execute from a non-`X` region raises an *instruction access fault*, reading from a non-`R` region raises a *load access fault*, and writing to a non-`W` region raises a *store access fault*. When such a trap occurs, the core writes `mcause` with two pieces of information: a flag indicating whether the trap is an interrupt, and a cause code identifying the specific reason. The handler uses `mcause` together with `mepc`, which holds the program counter at which the trap was taken, to determine the appropriate service routine and, if necessary, to adjust control flow by updating `mepc` to the following instruction to be executed. If `mepc` is not updated and the underlying condition is not corrected, the processor resumes at the same faulting instruction and immediately retriggers the exception, yielding a trap loop with no forward progress. The register width follows the implementation's XLEN, but for practical purposes, it is sufficient to note that the most significant bit of `mcause` distinguishes interrupts from exceptions and the remaining bits carry the cause code.

6.2.1.5 Representative exception causes (Ibex) In the context of PMP and secure boot evaluation, the following Ibex exception codes are the most relevant. The names and numeric values correspond to the `ibex_exc_t` enumeration used by the firmware.

1. **`kIbexExcInstrAccessFault` (1)** Raised when an instruction fetch targets an address without execute permission, for example, PMP denies `X` or otherwise cannot be accessed.

2. **`kIbexExcIllegalInstrFault` (2)** Raised when the fetched instruction encoding does not correspond to any valid opcode for the implemented ISA or is not permitted in the current context. In this work, it is common to execute from regions prefilled with all ones (for example, `0xFFFF...`). If execute permission is denied by PMP/ePMP, the trap observed is an *instruction access fault*; if execution is permitted, the all-ones word does not decode to a legal instruction and therefore raises an *illegal instruction* fault.

3. **`kIbexExcLoadAccessFault` (5)** Raised when a load violates access policy, for example, PMP denies `R` or targets an inaccessible region.

4. **`kIbexExcStoreAccessFault` (7)** Raised when a store/AMO violates access policy, for example, PMP denies `W` or targets an inaccessible region.

5. **`kIbexExcMax` (31)** Defines the maximum code value in the `ibex_exc_t` enumeration. It is not generated by hardware as a trap cause, rather it is typically used in firmware as a special "no exception recorded" marker.

Therefore, the `mcause` register provides a clear means of identifying which exception was raised by a given instruction fetch or data access during our experiments with PMP-configured regions. `kIbexExcInstrAccessFault` distinguishes *execute* permission failures from mere decode errors, that are captured by `kIbexExcIllegalInstrFault` when execution is permitted but the bits do not form a valid opcode, while `kIbexExcLoadAccessFault` and `kIbexExcStoreAccessFault` reveal *read* and *write* violations, respectively. In practice, this quartet lets us diagnose any kind of misconfigurations or find unexpected vulnerabilities. The `kIbexExcMax` value is not produced by hardware; it is used in firmware as a "no exception recorded" marker.

Looking ahead to testing, we will use these `mcause` outcomes as concrete pass/fail values when examining protection policies. For example, when execute permission is intentionally withheld from an unverified region, we expect `kIbexExcInstrAccessFault`, if execution is permitted over all-ones filler, we expect `kIbexExcIllegalInstrFault`. Likewise, read or write probes into protected areas should yield `kIbexExcLoadAccessFault` or `kIbexExcStoreAccessFault`. By comparing the observed to the expected `mcause` codes across our testing, we can validate secure boot assumptions, as well as analyse and learn from OpenTitan's PMP/ePMP layouts.

6.2.1.6 Summary and motivation for ePMP PMP lets machine mode partition the physical address space into regions with explicit `R/W/X` permissions using `pmpaddr` and `pmpcfg`. Entries are matched in index order (the lowest index wins). In base PMP, two behaviors create risk if the configuration is incomplete or wrong: (i) if an access does not match any entry, it is allowed by default for S/U, and (ii) M-mode ignores PMP completely unless an entry's

L bit is set. In practice, this means that a small mistake such as leaving a gap between regions (TOR boundary off by one), failing to set the L bit or simply not configuring a memory region can open "holes" where code is able to execute from unintended areas or where M-mode is able to rewrite the protections. These pitfalls are common during early boot, when only a few entries are programmed and the system is still running in M-mode.

The enhanced PMP (ePMP) extension addresses exactly these pain points. It introduces the mseccfg CSR to control machine-mode lockdown and default policy, enabling a true whitelist-by-default behavior and consistent enforcement in M-mode during secure boot. In the next subsection (6.3.2) we describe how ePMP's policy bits are used by the ROM to lock down execution, remove the default-allow gaps, and keep the configuration robust throughout the boot chain.

6.2.2 ePMP

6.2.2.1 Bridge and scope Building on the limitations outlined above, the enhanced PMP (ePMP, standardized as Smepmp[5]) introduces machine mode policy controls that enable whitelist by default behavior and reliable enforcement during secure boot, without permanently consuming extra PMP entries or relying on rule ordering.

6.2.2.2 The mseccfg CSR ePMP adds the machine-mode CSR mseccfg to control early-boot security policy. The register is accessible only in M-mode and defines policy bits that alter PMP's default behavior while remaining fully compatible with base PMP when these bits are left cleared. On Ibex/OpenTitan only a subset is used and the remaining bits are reserved.

6.2.2.3 Rule Locking Bypass (mseccfg.RLB) This bit provides a controlled, temporary override of the locked state so that boot code can safely evolve PMP after first constraining M-mode:

- **If RLB=1:** software may modify or remove entries even when pmpcfg.L=1, allowing the ROM to lock down early, verify the next stage, and then reconfigure cleanly.
- **If RLB=0 and any entry has L=1:** RLB is held at 0 and further writes to set it are ignored until a PMP reset. This prevents late-stage code from reopening a bypass once protections are committed. This stickiness is a defense-in-depth choice to avoid compromising earlier security features.

6.2.2.4 Machine-Mode Whitelist Policy (mseccfg.MMWP) This bit changes the default outcome for M-mode when no PMP entry matches:

- **If MMWP=1:** unmatched accesses are denied (true whitelist by default).
- **If MMWP=0:** base PMP semantics apply, therefore, unmatched accesses are not blocked by PMP (subject to L semantics).

In practice, platforms enable MMWP early or wire it through RTL configuration and leave it on through the secure-boot chain to avoid reintroducing default-allow windows.

6.2.2.5 Machine-Mode Lockdown (mseccfg.MML) When enabled, MML reinterprets the meaning of the pmpcfg.L bit to partition enforcement by privilege domain. Concretely, if pmpcfg.L == 1, the corresponding PMP entry is enforced only in M-mode (and is not applied to U/S mode). If pmpcfg.L == 0, the entry is enforced only in U/S-mode (and is ignored in M-mode). When MML == 0, base PMP semantics apply instead: L locks the entry and extends its enforcement to M-mode (otherwise M-mode ignores the entry). OpenTitan's standard flow does not rely on MML therefore we do not use it in our experiments. For the formal definition and additional edge-case behavior, refer to the Smepmp specification[5].

6.2.2.6 Why ePMP matters for OpenTitan During ROM and ROM_EXT, only a minimal verified ROM region should be executable, while all other regions should be non-executable. MMWP removes default-allow holes, while RLB gives the boot flow a safe way to lock early and still evolve the ePMP layout once the next stage is authenticated. This yields a more robust secure-boot posture than base PMP alone, while preserving compatibility (by leaving mseccfg cleared) for ablation tests and comparisons.

6.2.3 OpenTitan's ePMP configuration

In this section we analyse how OpenTitan configures ePMP across reset and early boot, highlighting the transition from a minimal reset initialization to the fully-fletched execution capable configuration after verification of the next stage.

6.2.3.1 Hardwired Configuration At reset, Ibex loads a minimal ePMP state from RTL constants. The arrays PmpCfgRst[16] and PmpAddrRst[16] predefine a small set of regions (ROM, MMIO, Debug ROM) while leaving the remainder *OFF*. The machine security configuration CSR resets to mseccfg.RLB=1 and mseccfg.MMWP=1. Thus, unmatched addresses are denied by default in M-mode, and the ROM is allowed to refine or remove locked entries during early boot. Leaving unused entries *OFF* is therefore safe: with MMWP=1 there are no default-allow gaps at reset.

[Maybe add Annex with RTL code?]

6.2.3.2 ROM's ePMP Init Table 2 summarizes the ROM-stage ePMP layout. Key regions include **ROM TEXT** (RX), **ROM** data (R), and **MMIO** (RW). Notably, the **eFlash** is mapped as Read-Only to allow signature verification, but remains non-executable to prevent premature code execution.

6.2.3.3 Allowing ROM_EXT's execution Once the ROM has validated the ROM_EXT's manifest and signature, it updates ePMP to permit execution of the ROM_EXT text only. Practically, the function rom_epmp_unlock_rom_ext_rx programs a *TOR* pair to bound **ROM_EXT TEXT** with RX and finally configures a *NAPOT* R region for **ROM_EXT** read-only sections.

TABLE 2: ROM-STAGE EPMP ENTRIES

Entry	Description	Permissions	Addressing Mode
1	ROM TEXT	RX	TOR
2	ROM	R	NAPOT
5	eFlash	R	NAPOT
11	MMIO	RW	TOR
14	Stack Guard	<none>	NA4
15	RAM	RW	NAPOT

ROM-stage ePMP entries programmed by ROM. ROM_EXT remains non-executable until explicitly unlocked after signature verification

During this step `mseccfg.RLB`=1 allows modification of entries that were previously locked. After hand-off, later stages can restrict reconfiguration by clearing/binding off that flexibility. This staged enablement ensures that flash becomes executable only after cryptographic verification, while the deny by default posture remains in effect for all unmatched regions.

7 EXPERIMENTAL METHODOLOGY

The experimental testing is conducted using Verilator, a cycle-accurate simulator that compiles the digital RTL design (SystemVerilog code) of OpenTitan's Earlgrey SoC into a high-performance C++ model. This approach enables efficient emulation of the hardware platform, offering a balance between execution speed and functional fidelity suitable for firmware-level security analysis.

7.1 Test environment

The OpenTitan development ecosystem provides a toolchain organized by **Bazel** to build both the hardware simulation and the firmware images. **FuseSoC** manages the RTL dependencies, which are translated by **Verilator** into a cycle-accurate C++ executable (`Vchip_sim_tb`). Finally, **Opentitantool** interfaces with this binary to load firmware images (ROM, Flash, OTP) and capture UART output during execution.

It is important to note that while Verilator is faster than traditional logic simulators, the cycle-accurate nature of the emulation still imposes significant computational overhead. Consequently, complex test sequences—particularly those involving cryptographic operations or extended boot flows—resulted in high simulation times, with some individual tests requiring upwards of fifteen minutes to complete in an 8-core Intel Xeon Gold 6130 CPU at 2.10GHz.

Regarding the experimental methodology, we utilize two distinct instances of this simulation environment. The first instance serves as the baseline, running the unmodified SoC design to establish expected behavior. The second instance is used for experimental ablation, where specific security policies are removed, relaxed, or altered to quantify their impact on the system's overall security posture.

An emulation approach using Field-Programmable Gate Arrays (FPGAs) was initially considered, however, given schedule constraints and late hardware availability, we considered the approach beyond the scope of this thesis.

7.2 Experimental Design And Implementation

To systematically verify memory protections, we cannot simply rely on standard software assertions, as a security violation triggers a hardware trap (exception) rather than a return code. OpenTitan addresses this with a baseline test, `rom_epmp_test.c`, which probes specific memory locations and compares the resulting exception with an expected value. However, this test is limited in scope, as it is constrained primarily to execute-only accesses and does not cover the full range of memory kinds and access types required for a complete security verification.

To overcome these limitations, we developed a custom test suite that operates as a direct replacement for the standard Mask ROM firmware. By linking against the ROM linker script and utilizing the standard reset vector, our test executes in Machine Mode (M-Mode) immediately following system reset. This privileged context is essential for verifying the initial ePMP configuration and the locking mechanisms. Instead of proceeding with the standard boot flow -which would attempt to verify and jump to the ROM_EXT in Flash- this test changes the control flow to execute our verification routines directly from the ROM text region.

Furthermore, for these experiments, the eFlash memory is left in its erased state (filled with `0xFFFFFFFF`). This provides a predictable target for testing invalid instruction execution, as such value is not a valid RISC-V instruction. Crucially, this also acts as a **fail-safe**: if a security gap were to erroneously permit execution in this region, the processor would immediately trap on the first instruction fetch rather than continuing to execute potentially undefined memory regions, ensuring that any failure is contained and observable. Finally, from a simulation perspective, this strategy reduces computational overhead by treating the flash as a constant value that the simulator can serve efficiently, therefore avoiding latency associated with loading large binary images or modeling complex memory content.

7.2.1 Custom Test Suite

To support this approach we adapted the existing test infrastructure to recover from intentional faults. While the standard OpenTitan exception handler is capable of recovering from execution faults it treats load or store exceptions as fatal errors. Consequently we expanded the handler logic to intercept these additional exception types and resume execution.

7.2.1.1 Fault Injection and Recovery We introduced two helper primitives, `read32` and `write32`, which attempt 32-bit accesses to a target address. Before executing the operation, these functions reset a global state variable, `exception_received`, to a sentinel value (`kIbexExcMax`). If the access triggers a hardware trap due to ePMP enforcement, control is automatically transferred to the exception handler.

The verification logic is encapsulated in a generic `TestAccess` routine (Algorithm 1). This routine attempts a specific memory operation (Read, Write, or Execute) using the helpers and verifies that the hardware response matches the expected security policy.

Algorithm 1 Memory Protection Verification Routine

```
1: Global volatile exception_received
2: procedure VERIFYREGION(base, size, op, expect)
3:     start ← base
4:     end ← base + size − 4
5:     for addr ∈ {start, end} do
6:         ACCESS(addr, op, expect)
7:     end for
8: end procedure
9: procedure ACCESS(addr, op, expect)
10:    exception_received ← kIbexExcMax
11:    try perform op (read / write / execute) on addr
12:    if Hardware Trap Occurs then
13:        ROM_EXCEPTION_HANDLER
14:    end if
15:    if exception_received ≠ expect then
16:        Report Failure
17:    end if
18: end procedure
19: procedure ROM_EXCEPTION_HANDLER
20:    mcause ← CSR_READ(mcause)
21:    exception_received ← mcause
22:    CSR_WRITE(mepc, next_instruction)
23: end procedure
```

7.2.1.2 Exception Handling The existing exception handler was extended to support load and store exceptions in addition to execution faults. Upon entry, the handler reads the `mcause` CSR to identify the exception type and records it in the global `exception_received` variable.

To allow the test to proceed, the handler must advance the Machine Exception Program Counter (`mepc`) to the next instruction. Crucially, since the Ibex core supports the RISC-V Compressed Extension (C), instructions may be either 16-bit or 32-bit wide. Simply incrementing the PC by a fixed 4 bytes resulted in misalignment or infinite trap loops. Debugging this was challenging since the failures were intermittent and depended on the specific instruction encoding. Therefore, the handler checks the lower bits of the faulting instruction to determine its length and increments `mepc` by either 2 or 4 bytes accordingly.

7.2.1.3 Region-Specific Verification Dedicated verification functions were implemented for each distinct memory region (e.g., ROM, eFlash, RAM, MMIO). Given that an exhaustive scan of the entire address space would be prohibitively slow in a cycle-accurate simulator like Verilator, our verification strategy focuses on critical addresses-specifically the start and end words of each defined region-where configuration errors are most likely to manifest.

Within these functions, the test logic probes these boundaries using the appropriate access helpers. The outcome is then validated against the hardcoded expected exception (e.g., `kIbexExcInstrAccessFault` for non-executable regions), and any discrepancy triggers a failure report. Additionally, specific functions were implemented to verify the "sticky" semantics of the `mseccfg` register, ensuring that critical security policies cannot be reverted by software once established.

7.2.1.4 Watchdog Management Finally, a mechanism to disable the watchdog timer was introduced to prevent unintended system resets during prolonged test execution. Unlike the original short-duration tests, our extended testing exceeded the default watchdog window. To debug the root cause of unexpected resets, the reset cause register was analyzed, indicating a reset triggered by an interrupt. Consequently, the watchdog is now explicitly disabled at the beginning of the test sequence to ensure stability.

7.2.2 Verification Strategy

Based on the described methodology, we defined a set of experiments to characterize the ePMP implementation. These tests are driven from the ROM/ROM_EXT context and validate four distinct security properties.

7.2.2.1 Access control for configured regions First, we validate the permissions for regions explicitly covered by ePMP entries (Table 2). The test issues *execute*, *load*, and *store* probes to both interior addresses and boundary words of each region.

To pass this verification, the hardware must strictly enforce the defined permissions. For example, the **FLASH** region is configured as Read-Only, therefore, the test expects load operations to succeed, while store and execute attempts must trigger `kIbexExcStoreAccessFault` and `kIbexExcInstrAccessFault`, respectively. Any deviation from this behavior is flagged as a security violation.

7.2.2.2 Default-deny for unconfigured regions (`MMWP=1`) Subsequently, we probed addresses falling outside any configured ePMP entry. To rigorously test the whitelist policy, we generated a build variant where the eFlash PMP rule was omitted, effectively leaving the entire flash storage unmapped.

The objective is to verify that with `MMWP=1`, the system behaves as a true whitelist. The test is considered successful only if instruction fetches, loads, and stores to this unmapped area consistently trigger access faults, thereby confirming that no "catch-all" deny rule is required to block undefined memory.

[Annex eFLASH unconfigured]

Figure 3 illustrates the logical flow of this verification process. The diagram presents a high-level abstraction using Flash memory accesses as a representative example, depicting a generic unconfigured region adjacent to the Flash to visualize the default-deny behavior. It is important to note that in the experimental implementation, this scenario was validated by explicitly omitting the eFlash PMP configuration. This effectively rendered the entire Flash window unconfigured, allowing us to confirm that the hardware enforces the whitelist policy (`MMWP=1`) and blocks access in the absence of a matching rule.

7.2.2.3 ROM_EXT transition: from read-only to executable The handover from ROM to ROM_EXT is a critical security boundary in OpenTitan's boot flow: it provides the first Root of Trust guarantee, it necessitates strict access control. ROM code must read and verify the ROM_EXT image signature before granting execution, therefore, reads are

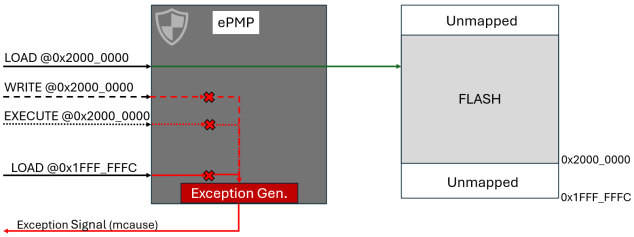

Fig. 3: High-level abstraction of the logical flow of the Region-Specific Verification. The ePMP unit acts as a gatekeeper. Valid accesses (Green) reach physical memory, while invalid accesses (Red) are blocked at the gate and routed to the Exception Generator, which signals the trap (`mcause`) back to the core.

required for verification, but execution must remain blocked until verification completes.

To validate this behavior, we define two checkpoints. First, the test asserts that the ROM_EXT region is initially configured as non-executable (read-only). Second, after simulating a successful verification, the test applies the ROM's ePMP update and asserts that execution permissions are correctly granted. This ensures that privileges are contingent upon a deliberate configuration change.

7.2.2.4 Effect of RLB on locked entries

Lastly, the Rule-Locking Bypass test verifies that the `mseccfg.RLB` bit correctly governs the mutability of locked PMP entries. The test sequence attempts to modify a locked entry (e.g., changing ROM_EXT permissions) under two conditions: first with `RLB`=1 (expecting success), and subsequently with `RLB`=0 (expecting failure). This validates that clearing `RLB` effectively renders locked entries immutable.

7.3 Ablation study: Disabling ePMP

(show how ePMP was removed)

To quantify the specific security benefits of ePMP, we established a baseline that mimics standard PMP semantics (absence of enhanced features). We achieved this by modifying the hardware configuration at the RTL level to hardwire the reset values of the `mseccfg` register to zero. This effectively disables the Machine-Mode Whitelist Policy (MMWP) and Rule Locking Bypass (RLB).

We then executed the exact same verification suite described in Section 7.2.2 on this degraded target. By comparing the exception responses of the standard OpenTitan configuration against this PMP-only baseline, we isolate the guarantees provided by the extension. We focus on two key behaviors:

- **Unconfigured Regions (MMWP=0):** We repeat the eFlash probes to verify if the default-deny policy holds when the whitelist feature is disabled. We expect accesses to unmapped regions to succeed (default-allow) rather than trap.
- **Locked Regions (RLB=0):** We re-evaluate the ROM region permissions to determine if the lack of the Rule Locking Bypass affects the firmware's ability to refine permissions during boot. We specifically test whether the ROM data section can be successfully locked down to Read-Only after the initial boot sequence.

8 Results & Conclusion

8.1 Baseline Security Verification (ePMP Enabled)

The verification strategy defined in Section 7.2.2 was first executed against the standard OpenTitan configuration.

- **Configured Regions:** All regions behaved as expected. The Flash region correctly rejected execution attempts with `kIbexExcInstrAccessFault`.
- **Whitelist Policy:** The unmapped Flash test confirmed the `MMWP=1` policy, where all accesses were blocked by hardware.
- **Handover:** The ROM_EXT transition test confirmed that code execution is impossible until the explicit unlock occurs.

8.2 Impact of ePMP Removal (Ablation Study)

When running the verification suite on the ablation target (base PMP only), two critical security regressions were observed.

8.2.1 Failure of Default-Deny Policy

With `MMWP` hardwired to 0, our probes to unconfigured memory regions (specifically the unmapped eFlash) completed without raising exceptions. This confirms that without ePMP, the system defaults to a "blacklist" approach, where any memory not explicitly covered by a PMP entry remains accessible to Machine Mode.

As illustrated in Figure 4, under this configuration, accesses to the unconfigured area bypass the protection logic and complete successfully, standing in clear contrast to the default-deny behavior observed when `MMWP` is enabled.

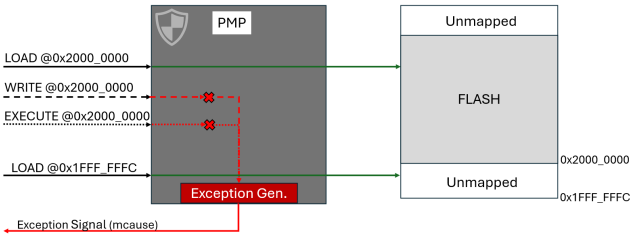

Fig. 4: High-level abstraction of the logical flow of the Region-Specific Verification, now showcasing PMP behaviour with `MMWP` = 0.

8.2.2 Inability to Harden ROM Permissions

During the execution of the ablation test suite, a critical anomaly was detected in the ROM region verification. While most security checks aligned with expectations for a PMP-only system, the test case asserting that the ROM read-only data section (`.rodata`) must be non-executable

failed consistently. Contrary to the secure boot policy, the hardware permitted code execution from this data-only region.

Initially suspected to be a software bug in the test suite, further investigation revealed that this behavior was a direct architectural consequence of disabling ePMP. At reset, the hardware loads a default PMP entry covering the entire ROM address space with Locked, Read, and Execute (L/R/X) permissions.

This permissive default is architecturally necessary for two reasons. First, the processor must be able to fetch the reset vector immediately upon power-on, otherwise, an instruction access fault would occur before the first instruction is executed. Second, while the hardware knows the physical size of the ROM, it is agnostic to the software layout. The boundary between executable code (`.text`) and constant data (`.data, .rodata` among others), is determined at compile time. Consequently, the RTL cannot enforce a granular restriction without risking compatibility with future firmware builds, forcing it to map the entire ROM size as executable.

In the standard boot flow, the `rom_epmp_init` function resolves this by reading the firmware's linker symbols to identify the exact code boundary. It then attempts to remove the execute permission from the original wide entry and add a new, granular rule granting execution only to the `.text` section.

However, with `RLB` forced to 0 in our ablation study, the initial "wide" PMP entry loaded at reset is immediately and permanently locked. Consequently, the firmware's attempt to restrict the data section to Read-Only is blocked by hardware. Our tests confirmed that the ROM data section remained executable throughout the test, exposing a potential attack surface for Return-Oriented Programming (ROP) attacks. This finding highlights that `RLB` is a necessity for transitioning from a permissive boot state to a secure runtime configuration.

8.3 Identified Attack Vector: The Initialization Gap

The results from the ablation study reveal a critical attack vector that emerges during the early boot phase. Specifically, a significant window of vulnerability exists between the hardware reset and the completion of the `rom_epmp_init` function, which is responsible for configuring the full PMP layout.

During this interval, the system relies on the minimal PMP configuration hardwired at reset, which leaves large portions of the memory map - including the eFlash - unconfigured. As demonstrated in our previous testing, when `MMWP`=0 base PMP semantics apply, therefore, Machine Mode is permitted to execute from any address not explicitly covered by a PMP entry. This creates a substantial risk: an attacker could induce a control-flow glitch (eg., via voltage fault injection, as outlined in our Threat Model) to redirect the Program Counter (PC) to the unmapped eFlash region before initialization completes, the processor will successfully execute the code found there, even if it is malicious. This effectively bypasses the entire Secure Boot chain, allowing the execution of arbitrary, unverified payloads before the Root of Trust has established its defenses.

ACKNOWLEDGEMENTS

[TBD]

REFERENCES

[1] *OpenTitan: Open Source Silicon Root of Trust*, Disponible: https://opentitan.org

[2] A. Meza, C. Tunc, P. Nasahl, and J. Grossklags, "Security Verification of the OpenTitan Hardware Root of Trust," doi: 10.1109/MSEC.2023.3255363.

[3] A. Musa, E. Parisi, L. Barbierato, A. Acquaviva, and F. Barchi, "End-to-end Integration of OpenTitan Security Features in a Pure RISC-V SoC," doi: 10.1109/SMACD61181.2024.10745397.

[4] S. K. Saha, A. N. Butka, M. K. Ahmed, and C. Bobda, "OpenTitan based Multi-Level Security in FPGA System-on-Chips," doi: 10.1109/ICFPT59805.2023.00056.

[5] *Secure Memory Protection Extension (PDF)*, Github: riscvarchive/riscv-tee Project. Disponible: `https://github.com/riscvarchive/riscv-tee/blob/191b563b08b31cc2974d604a3b670d8666a2e093/Smepmp/Smepmp.pdf` (Accedit: 2025-09-23)

[6] *The RISC-V Instruction Set Manual (Unprivileged ISA)*. RISC-V International. Disponible: `https://riscv.org/technical/specifications/` (Accedit: 2025-11-15)

[7] *Hardware Root of Trust*. Rambus Blog. Disponible: `https://www.rambus.com/blogs/hardware-root-of-trust/` (Accedit: 2025-11-15)
Keystone: An Open Framework for Architecting Trusted Execution Environments. EuroSys 2020. Disponible: `https://www.shwetashinde.org/publications/keystone_eurosys20.pdf` (Accedit: 2025-11-15)

[8] *RISC-V Memory Protection: Diving deep into the complexities*. Medium (Karthik B K). Disponible: `https://medium.com/@talktokarthikbk/risc-v-memory-protection-diving-deep-into-the` (Accedit: 2025-11-15)

[9] *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture (inclou PMP)*. RISC-V International. Disponible: `https://github.com/riscv/riscv-isa-manual/releases/download/Priv-v1.12/riscv-privileged-20211203.pdf` (Accedit: 2025-11-15)

[10] *Verified Boot Crypto*. Chromium OS Design Docs. Disponible: `https://www.chromium.org/chromium-os/chromiumos-design-docs/verified-boot-crypto/` (Accedit: 2025-11-15)

[11] *OpenTitan Partnership Makes History: First Open-Source Silicon to Reach Commercial Availability*. lowRISC News (13 Feb 2024). Disponible: https://lowrisc.org/news/opentitan-commercial-availability/ (Accedit: 2025-10-04)

[12] *"A booting computer is as vulnerable as a newborn baby"*. ETH Zürich News (5 Nov 2019). Disponible: https://ethz.ch/en/news-and-events/eth-news/news/2019/11/project-opentitan.html (Accedit: 2025-11-15)

[13] *Nuvoton Develops OpenTitan®-based Security Chip as Commercial Product*. Press Release (30 May 2024). Disponible: https://www.nuvoton.com/news/news/all/TSNuvotonNews-000514/ (Accedit: 2025-11-15)

[14] *Fabrication begins for production OpenTitan silicon*. Google Open Source Blog (6 Feb 2025). Disponible: https://opensource.googleblog.com/2025/02/fabrication-begins-for-production-opentitan-silicon.html (Accedit: 2025-11-15)

[15] *Seagate Designs RISC-V Cores to Power Data Mobility and Trustworthiness*. Seagate Article. Disponible: https://www.seagate.com/stories/articles/seagate-designs-risc-v-cores-to-power-data-mobility-and-trustworthiness-master-pr/ (Accedit: 2025-11-15)

[16] *Western Digital Collaborates with lowRISC and Google to Increase Security Transparency in Data-Centric Platforms*. Western Digital Press Release (5 Nov 2019). Disponible: https://www.westerndigital.com/company/newsroom/press-releases/2019/2019-11-05-western-digital-collaborates-with-lowrisc-and-google-to-increase-security-t (Accedit: 2025-11-15)

[17] *OpenTitan production silicon and early integrations*. lowRISC News (20 Feb 2025). Disponible: https://lowrisc.org/news/the-worlds-first-open-source-security-chip-hits-production-with-google/ (Accedit: 2025-11-15)

[18] *Hardware Trojans in Chips: A Survey for Detection and Prevention*. Sensors (MDPI), vol. 20, no. 18, 5165 (2020). Disponible: https://www.mdpi.com/1424-8220/20/18/5165 (Accedit: 2025-11-15)

[19] *OpenTitan Lightweight Threat Model*. Project documentation. Disponible: https://opentitan.org/book/doc/security/threat_model/index.html (Accedit: 2025-11-15)

[20] *Design Approaches and Architectures of RISC-V SoCs (referència a Sv32)*. RISC-V International Blog. Disponible: https://riscv.org/blog/design-approaches-and-architectures-of-risc-v-socs/ (Accedit: 2025-11-15)

APPENDIX